SDA WEEK 12 LEC 2

DEPLOYMENT DIAGRAMS

Syed Ahmed Khan

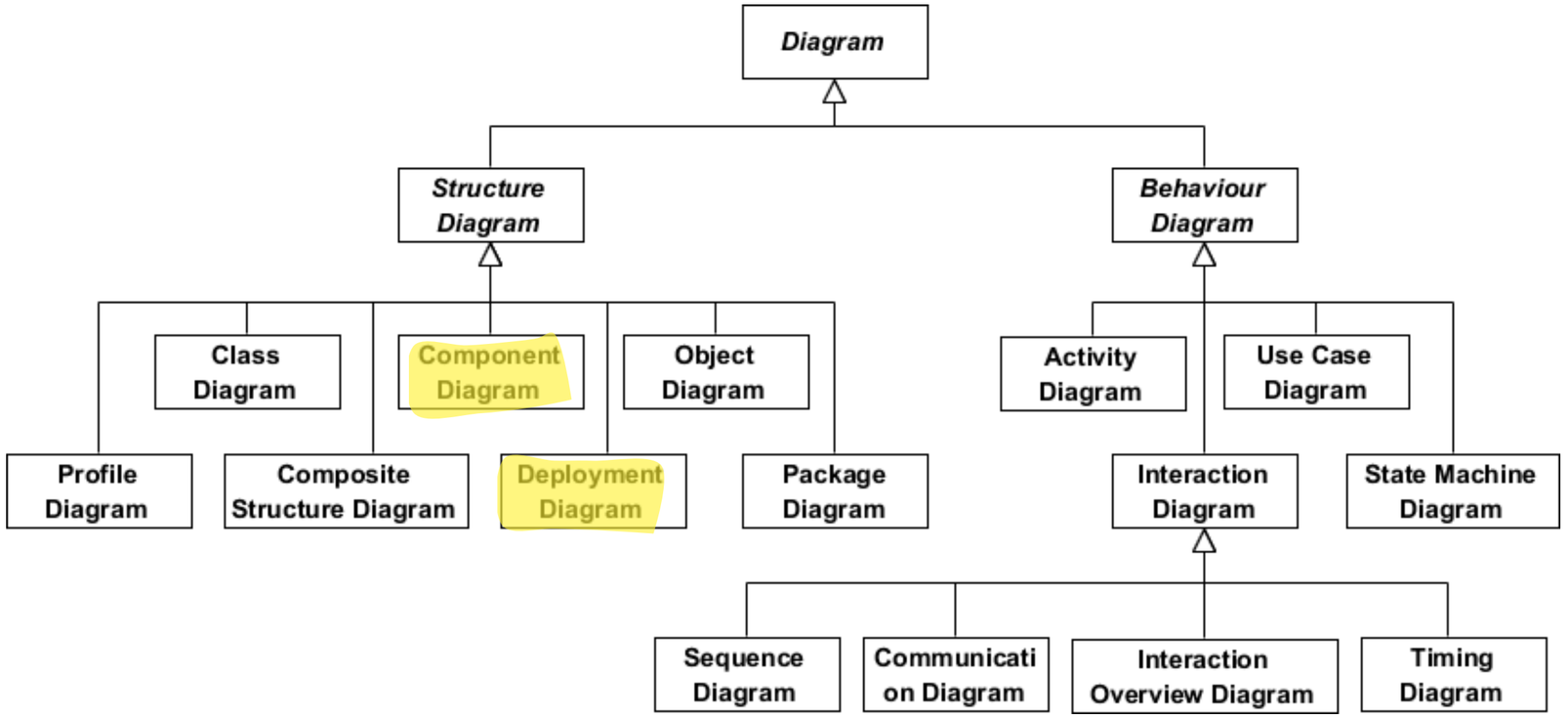
syed.ahmed@nu.edu.pk

RECAP

Implementation Diagrams:

- Represent the physical realization of software.
- Capture how the logical design (classes, interactions) is implemented in terms of components and nodes.
- Two major UML implementation diagrams:
  a. **Component Diagram:** Focuses on software structure
  b. **Deployment Diagram:** Focuses on hardware & runtime structure.

UML DIAGRAMS



RECAP : COMPONENT DIAGRAMS

Component Diagrams:

- A component is a self-contained unit of software that encapsulates functionality and provides interfaces for interaction.
- Example:
  - In a Banking System, a Transaction Processor, Authentication Module, and Database Access Layer can each be components.

RECAP : COMPONENT DIAGRAMS

- When you design a component diagram, you’re essentially designing how code modules communicate.
- You can think of it as your ‘plug-and-play’ view – change the database driver component, and your system still runs as long as the provided and required interfaces are identical.

TRANSITION: FROM LOGICAL TO PHYSICAL VIEW

Now that we know how components interact logically, the next question is where do these components live? Do they all run on one computer? Do some run on a web server while others on a database server? That’s exactly where Deployment Diagrams are used to model.

DEPLOYMENT DIAGRAM

A Deployment Diagram represents the physical architecture of a system, showing how software components (artifacts) are deployed on hardware nodes and how those nodes are interconnected.

Deployment diagrams bring the software into real world by showing how software gets assigned to hardware and how the pieces communicate.

DEPLOYMENT DIAGRAM

Imagine our online library system.

- The user interface runs in a browser on a client machine.
- The web application runs on an application server.
- The data is stored on a database server.
- This physical relationship, who runs where, is captured by the deployment diagram.

DEPLOYMENT DIAGRAM PURPOSE

- Show **physical layout** of hardware and software.
- Visualize **runtime structure** of a system.
- Identify **communication paths** (TCP/IP, HTTP, sockets).
- Help with **system configuration, scaling,** and **maintenance.**

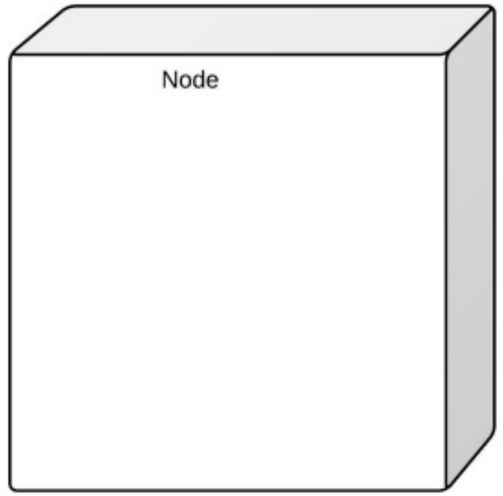
DEPLOYMENT DIAGRAM PURPOSE

These diagrams bridge your design model to the real infrastructure they help system architects and DevOps teams deploy and maintain systems efficiently.

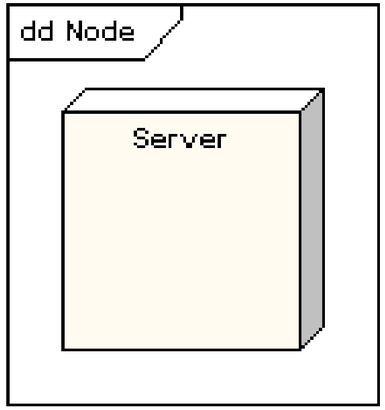
ELEMENTS OF DEPLOYMENT DIAGRAM

NODES

- Nodes Represent hardware or execution environments.
- Shown as a 3D box.
- Examples:
  - Hardware nodes: Server, PC, Router
  - Execution nodes: Web server, JVM, OS



NODES

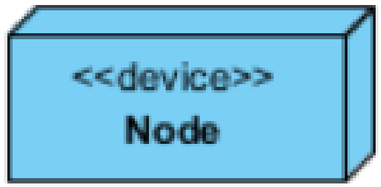


- One can think of nodes as the physical hosts or containers that execute components.

- A node may contain other nodes, for example, a Web Server Node might host multiple Application Containers.

HARDWARE NODES

- These are the actual physical machines. We stereotype them as <<device>>:
  - <<device>> Web Server
  - <<device>> Database Server
  - <<device>> Client Workstation
  - <<device>> Mobile Device



EXECUTION ENVIRONMENT NODES

- Execution environments are software platforms that run on hardware:
  - <<executionEnvironment>> Apache Tomcat
  - <<executionEnvironment>> J2EE Container
  - <<executionEnvironment>> .NET Runtime
  - <<executionEnvironment>> Docker Container

COONNECTIONS

- Represent communication paths between nodes.
- Can be labeled with protocols (TCP/IP, HTTP, RMI, etc.).
- Often drawn as solid lines connecting 3D boxes.

ARTIFACT

- Artifacts are physical manifestations of components discussed in the last lecture.
- Represent physical files: compiled code, executables, configuration, databases.
- Shown as rectangles with «artifact» stereotype and a document icon.

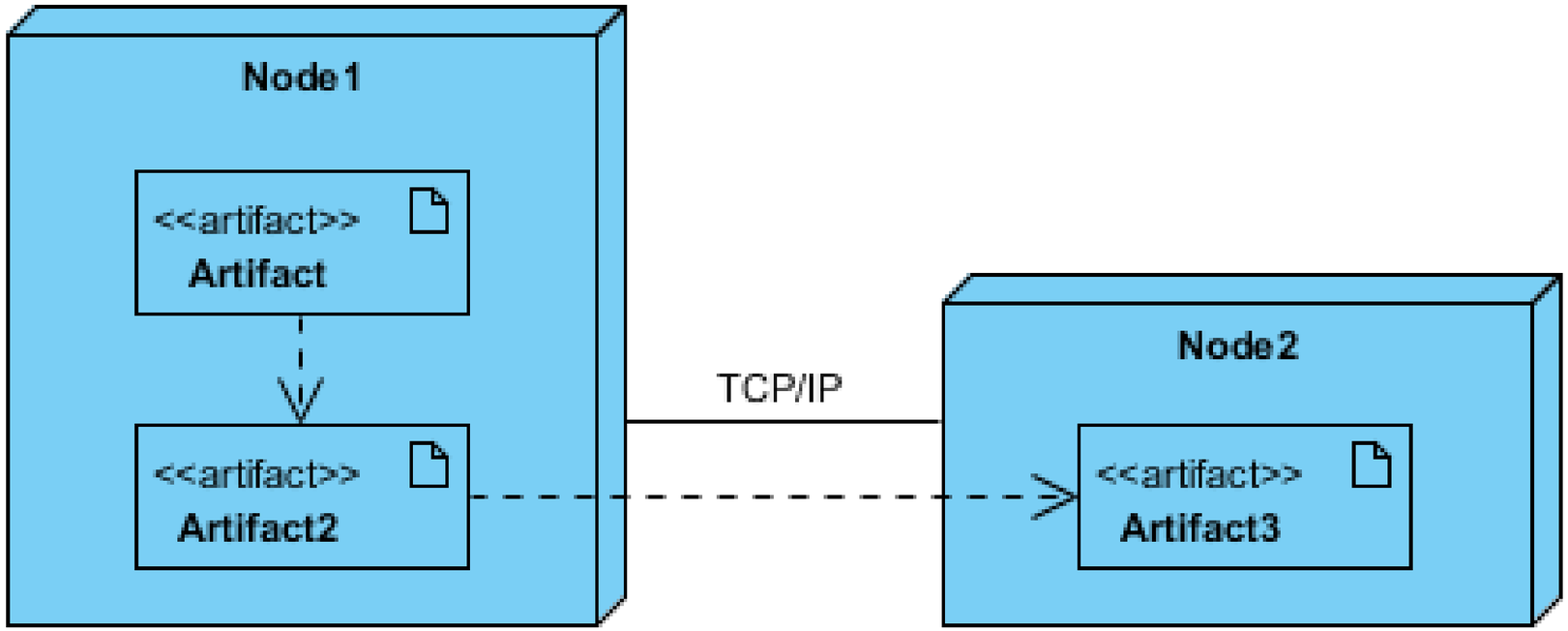
ARTIFACT

Artifacts are the ACTUAL files we deploy:

- .jar, .war, .ear files in Java
- .exe, .dll files in Windows
- Database script files
- Configuration files
- Examples:
  - «artifact» library.jar
  - «artifact» index.html
  - «artifact» config.xml

ARTIFACT


<<artifact>>
Artifact


Node1
<<artifact>>
Artifact
<<artifact>>
Artifact2
TCP/IP
Node2
<<artifact>>
Artifact3

WHEN DO WE USE DEPLOYMENT DIAGRAMS?

At the early stage: helps figuring out the general configuration.

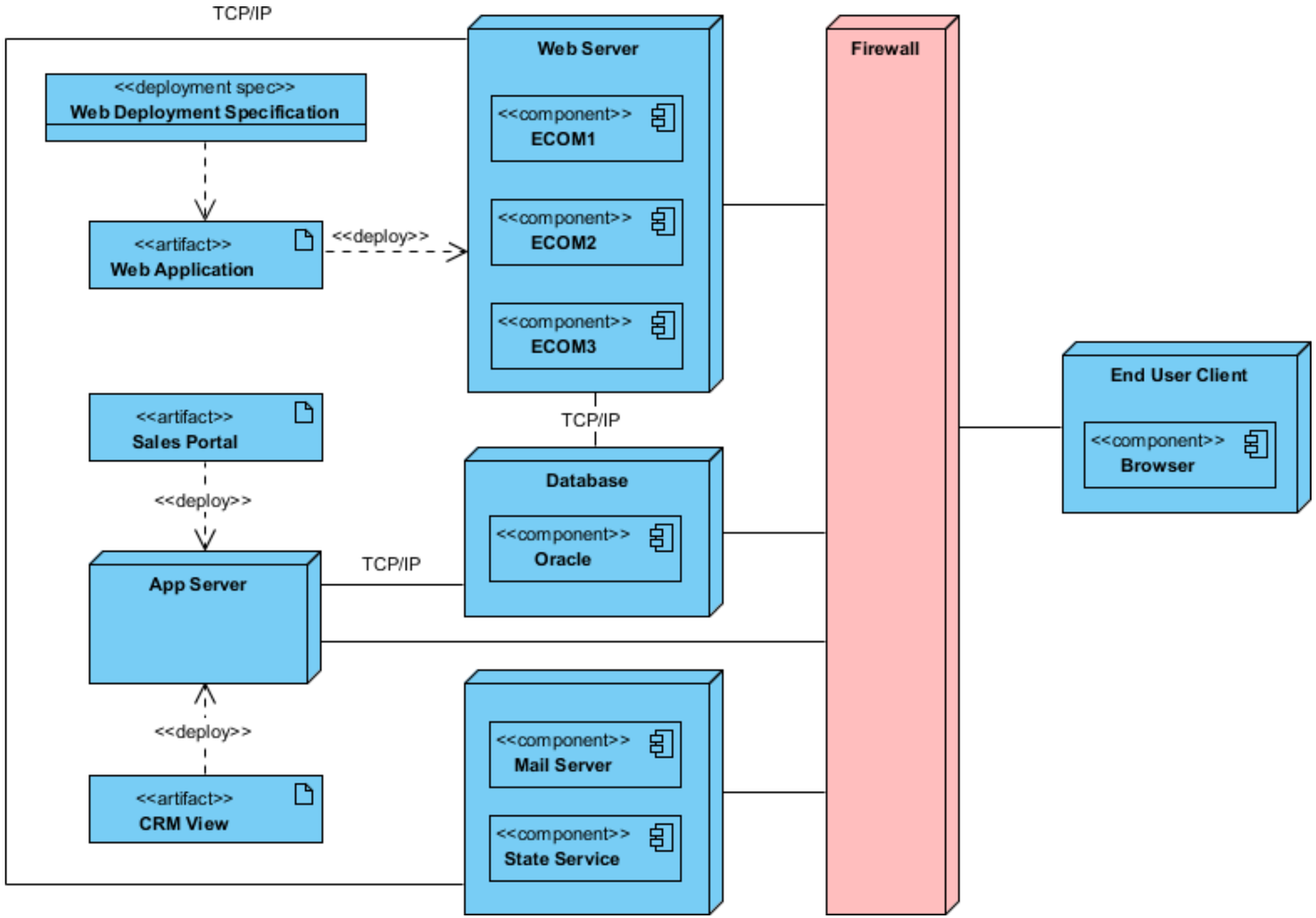
- Example: a web application will include:
  - A web server, application server and database
  - Clients access the application through browsers
  - The web server should have a firewall

WHEN DO WE USE DEPLOYMENT DIAGRAMS?

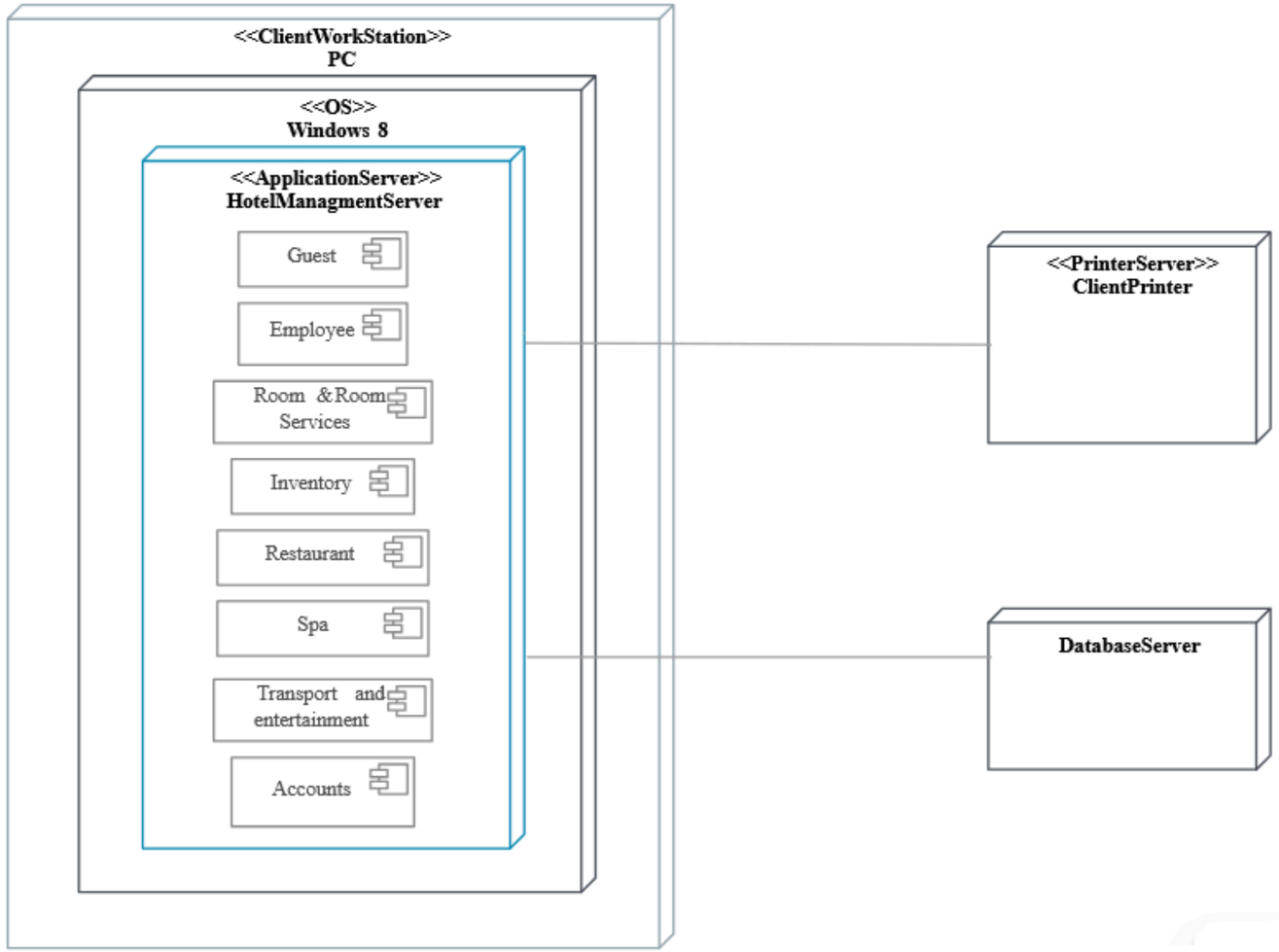
At the later stages: the deployment diagram will come into details about the system architecture.

- Which technology is used
- What communication protocols areused
- Software artifacts
- etc.

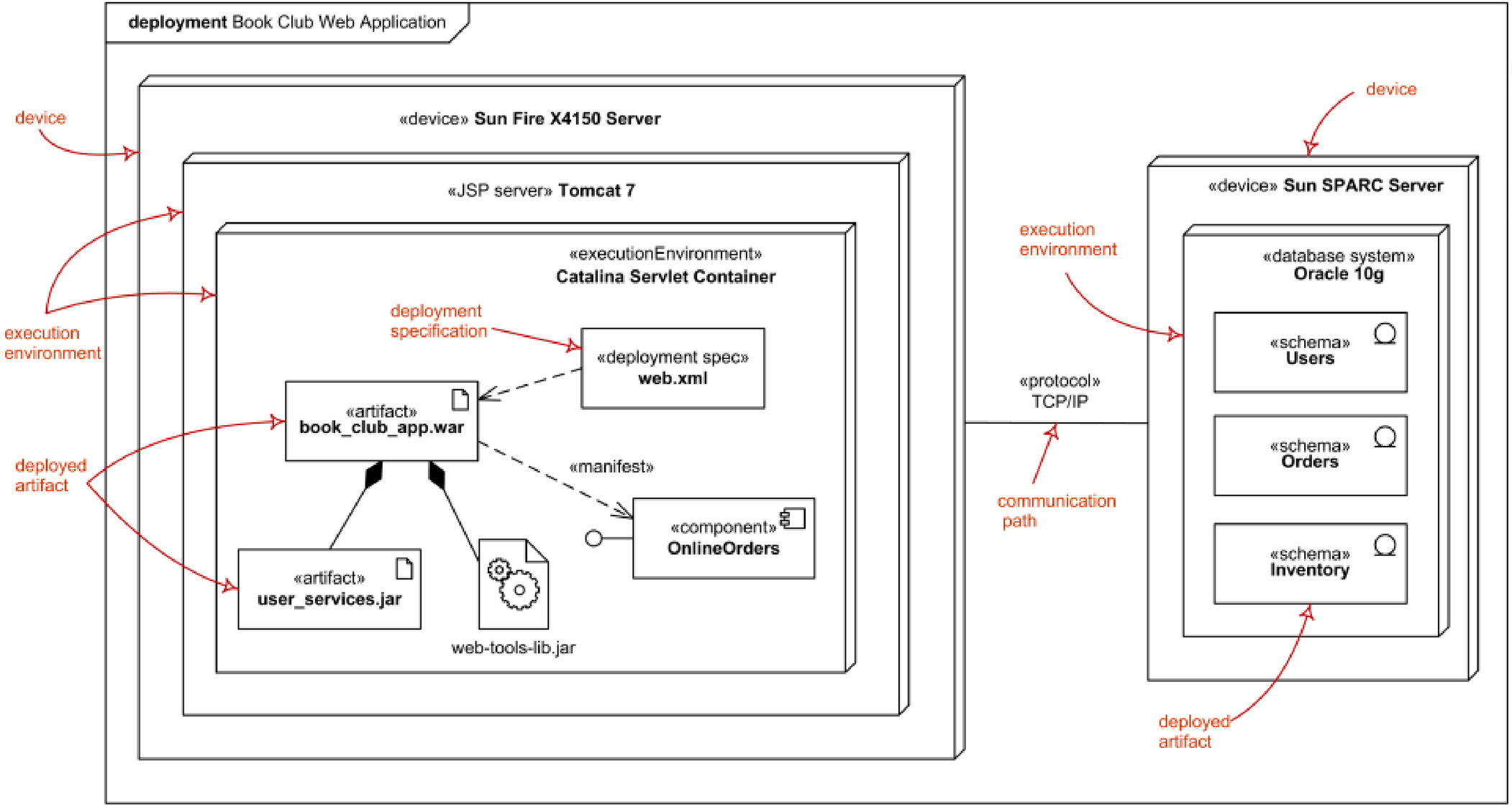
DEPLOYMENT DIAGRAMS EXAMPLE



DEPLOYMENT DIAGRAMS EXAMPLE



DEPLOYMENT DIAGRAMS EXAMPLE



THANK YOU